

AI READY PREMIUM KIT

Prompt Pack

202 prompt pronti per studiare Python con l'aiuto dell'AI

Ogni prompt con obiettivo, testo pronto, quando usarlo e cosa controllare

Materiale bonus di "Python da Zero con Google Colab" - Tania Cilio

Come usare il Prompt Pack

Saper chiedere all'AI è una competenza, e fa una differenza enorme. Una domanda vaga produce una risposta inutile; una domanda precisa produce un aiuto vero. Questo Prompt Pack ti dà 202 richieste già pronte, collaudate, per ogni situazione in cui un principiante usa l'AI mentre impara Python.

Le parti tra parentesi angolari, come <argomento> o <codice>, sono segnaposto: sostituiscili con la tua situazione concreta prima di inviare il prompt. Per ogni prompt trovi l'obiettivo (a cosa serve), il testo pronto, quando usarlo e - fondamentale - cosa controllare nella risposta dell'AI.

Regola d'oro, ripetuta in tutto il kit: prima provi tu, poi chiedi all'AI. E non usare mai codice che non capisci. Questi prompt servono a farti capire di più, non a saltare lo studio.

Capire un concetto da zero

1. Spiegazione semplice di un concetto

"Sono un principiante assoluto di Python. Spiegami <concetto> con parole semplici, senza dare per scontato nulla, e con un esempio della vita quotidiana."

Quando usarlo: Quando incontri un termine nuovo che non capisci.

Cosa controllare: Che la spiegazione non usi altri termini tecnici non spiegati. Se lo fa, chiedi di spiegare anche quelli.

2. Analogia per fissare un concetto

"Spiegami <concetto> in Python usando un'analogia concreta (per esempio un cassetto, una ricetta, una scatola). Voglio capire l'idea, non la definizione tecnica."

Quando usarlo: Quando la definizione tecnica non ti entra in testa.

Cosa controllare: Che l'analogia sia coerente: chiediti se regge fino in fondo o solo all'inizio.

3. Differenza tra due concetti simili

"Qual è la differenza tra <concetto A> e <concetto B> in Python? Spiegamela da principiante, con un esempio per ciascuno e quando usare l'uno o l'altro."

Quando usarlo: Quando due cose ti sembrano uguali (es. lista e tupla, = e ==).

Cosa controllare: Che dia un criterio chiaro per scegliere tra i due, non solo le definizioni.

4. A cosa serve davvero

"A cosa serve <concetto> nella pratica? Dammi 3 esempi reali in cui un principiante lo userebbe davvero."

Quando usarlo: Quando studi una cosa ma non capisci perché dovrebbe servirti.

Cosa controllare: Che gli esempi siano alla tua portata e non scenari troppo avanzati.

5. Spiegazione a livelli

"Spiegami <concetto> a tre livelli: prima in una frase per un bambino, poi in un paragrafo per un principiante, poi con un esempio di codice."

Quando usarlo: Quando vuoi capire un concetto in profondità, partendo dal semplice.

Cosa controllare: Che il livello 'principiante' sia davvero comprensibile prima di passare al codice.

6. Perché funziona così

"Perché in Python <comportamento>, es. la divisione / dà sempre un decimale? Spiegami il motivo in modo semplice."

Quando usarlo: Quando un comportamento ti sembra strano o controintuitivo.

Cosa controllare: Che la spiegazione abbia senso: se resta oscura, chiedi un esempio concreto.

7. Glossario personale

"Fammi un mini-glossario dei 5 termini più importanti legati a <argomento> in Python, ognuno con una definizione di una riga per principianti."

Quando usarlo: All'inizio di un nuovo argomento, per orientarti sul vocabolario.

Cosa controllare: Che le definizioni siano brevi e senza altri termini tecnici non spiegati.

8. Mappa dell'argomento

"Dammi una mappa semplice di cosa devo sapere su <argomento> in Python, dal più facile al più difficile, come una lista di tappe."

Quando usarlo: Quando inizi un argomento e non sai da dove partire.

Cosa controllare: Che le tappe partano davvero dalle basi e non saltino passaggi.

9. Esempio minimo

"Dammi l'esempio più piccolo e semplice possibile che mostra <concetto> in Python, con un commento su ogni riga."

Quando usarlo: Quando vuoi vedere un concetto isolato, senza distrazioni.

Cosa controllare: Che l'esempio sia davvero minimo: se è lungo, chiedine uno più corto.

10. Verifica di comprensione

"Ti spiego <concetto> con parole mie: <tua spiegazione>. È corretto? Dimmi cosa ho capito bene e cosa ho sbagliato."

Quando usarlo: Dopo aver studiato, per verificare di aver capito davvero.

Cosa controllare: Che corregga gli errori in modo specifico, non solo 'bravo' o 'sbagliato'.

11. Riassunto di fine capitolo

“Riassumi i punti chiave di <argomento> in Python in 5 frasi, come ripasso per un principiante.”

Quando usarlo: Alla fine di un capitolo, per fissare i concetti.

Cosa controllare: Che i punti siano effettivamente i più importanti e non dettagli secondari.

12. Domande per autovalutarsi

“Fammi 5 domande di comprensione su <argomento> in Python (solo domande, senza risposte) per verificare se l'ho capito.”

Quando usarlo: Dopo lo studio, per metterti alla prova.

Cosa controllare: Che le domande coprano il concetto centrale, non solo dettagli marginali.

13. Errori di comprensione tipici

“Quali sono i 3 fraintendimenti più comuni dei principianti su <concetto> in Python? Spiegameli così li evito.”

Quando usarlo: All'inizio di un argomento ostico, per non cadere nelle trappole.

Cosa controllare: Che siano fraintendimenti reali e spiegati con un esempio.

14. Collegamento con ciò che già so

“Ho già capito <concetto noto>. Spiegami <concetto nuovo> collegandolo a quello che già so.”

Quando usarlo: Quando vuoi costruire sul già noto invece di partire da zero.

Cosa controllare: Che il collegamento sia corretto e non forzato.

15. Quando NON usarlo

“Quando NON dovrei usare <concetto/strumento> in Python? Dammi i casi in cui è la scelta sbagliata.”

Quando usarlo: Quando hai imparato uno strumento e vuoi capirne i limiti.

Cosa controllare: Che dia casi concreti, non solo 'dipende'.

Sbloccare un errore

16. Capire un messaggio di errore

“Sono principiante. Ho questo errore in Python: <incolla il messaggio completo>. Cosa significa e come lo risolvo? Spiegamelo in modo semplice.”

Quando usarlo: Ogni volta che ottieni un messaggio rosso che non capisci.

Cosa controllare: Che spieghi la causa, non solo la soluzione: così impari a riconoscerlo.

17. Errore con codice allegato

“Questo codice dà errore. Ecco il codice: <codice>. Ecco l'errore: <errore>. Dimmi dov'è il problema e come correggerlo, spiegando il perché.”

Quando usarlo: Quando hai sia il codice sia il messaggio di errore.

Cosa controllare: Che la correzione sia minima e spiegata; prova sempre il codice corretto.

18. Errore senza messaggio (risultato sbagliato)

“Questo codice non dà errore ma il risultato è sbagliato. Mi aspetto <risultato atteso>, ottengo <risultato ottenuto>. Codice: <codice>. Dove sbaglio?”

Quando usarlo: Quando il programma gira ma il risultato non torna.

Cosa controllare: Che individui l'errore logico e spieghi perché produce quel risultato.

19. Tradurre l'errore in italiano semplice

“Traducimi questo errore Python in italiano semplice, come se lo spiegassi a chi non ha mai programmato: <errore>.”

Quando usarlo: Quando il messaggio in inglese ti spaventa.

Cosa controllare: Che la traduzione sia comprensibile e indichi cosa controllare.

20. Errori più probabili

“Il mio programma dovrebbe <obiettivo> ma non funziona. Quali sono i 3 errori più probabili per un principiante in un caso come questo?”

Quando usarlo: Quando non hai un messaggio chiaro e non sai dove guardare.

Cosa controllare: Che i 3 errori siano verificabili uno per uno nel tuo codice.

21. Checklist anti-errore

“Sto per scrivere codice che <obiettivo>. Dammi una checklist di 5 controlli per non fare gli errori tipici dei principianti.”

Quando usarlo: Prima di scrivere, per prevenire gli errori comuni.

Cosa controllare: Che i controlli siano concreti e applicabili, non generici.

22. Perché questo errore proprio qui

“Perché ottengo <tipo di errore> proprio su questa riga: <riga>? Spiegamelo da principiante.”

Quando usarlo: Quando l'errore punta a una riga che ti sembra corretta.

Cosa controllare: Che spieghi cosa Python si aspettava in quel punto.

23. Errore di indentazione

“Ho un errore di indentazione in questo codice: <codice>. Mostrami la versione con l'indentazione corretta e spiegami la regola.”

Quando usarlo: Quando ottieni IndentationError o TabError.

Cosa controllare: Che usi 4 spazi coerenti; controlla di non mischiare tab e spazi.

24. Errore di tipo (TypeError)

“Ho un TypeError in questo codice: <codice>. Quali tipi sto mescolando e come li converto correttamente?”

Quando usarlo: Quando sommi o confronti cose di tipo diverso.

Cosa controllare: Che indichi dove serve str(), int() o float() e perché.

25. Spiega e previeni

“Ho risolto questo errore: <errore>. Ora spiegami come evitarlo in futuro e come riconoscerlo subito la prossima volta.”

Quando usarlo: Dopo aver risolto, per non ripeterlo.

Cosa controllare: Che dia un segnale pratico per riconoscerlo in anticipo.

26. Errore in Colab specifico

“In Google Colab ottengo <comportamento strano, es. 'name not defined' su una variabile che ho creato>. Può dipendere dall'ordine delle celle? Spiegami.”

Quando usarlo: Quando l'errore in Colab sembra non avere senso.

Cosa controllare: Che consideri l'ordine di esecuzione delle celle: prova a riavviare il runtime.

27. Differenza tra errore e avviso

“Cosa distingue un errore (che blocca) da un avviso (warning) in Python? Il mio è <incolla>: è bloccante?”

Quando usarlo: Quando vedi messaggi ma il programma sembra funzionare lo stesso.

Cosa controllare: Che chiarisca se devi intervenire o puoi ignorarlo.

28. Ridurre il codice per trovare il bug

“Questo codice lungo dà errore: <codice>. Aiutami a ridurlo al pezzo minimo che riproduce il problema, così capisco dov'è.”

Quando usarlo: Quando il codice è lungo e non trovi il punto rotto.

Cosa controllare: Che il pezzo ridotto mostri ancora l'errore; è una tecnica di debug utile.

29. Spiega l'errore a parole tue

“Ti spiego cosa penso causi questo errore: <tua ipotesi>. Ho ragione? Se no, qual è la vera causa?”

Quando usarlo: Quando hai un'ipotesi e vuoi verificarla.

Cosa controllare: Che confermi o smentisca con una spiegazione, non solo sì/no.

30. Errore di valore (ValueError)

“Ho un ValueError convertendo un input: <codice>. Come gestisco il caso in cui l'utente scrive qualcosa di non valido?”

Quando usarlo: Quando int()/float() falliscono su input utente.

Cosa controllare: Che proponga try/except ValueError e tu capisca come funziona.

Far spiegare il codice

31. Spiegazione riga per riga

“Spiegami questo codice Python riga per riga, come se non avessi mai programmato: <codice>.”

Quando usarlo: Quando trovi un codice (in un libro, online, dall'AI) e non lo capisci.

Cosa controllare: Che ogni riga sia spiegata davvero; se ne salta qualcuna, richiedila.

32. Cosa fa in una frase

"In una sola frase semplice, cosa fa questo codice? <codice>"

Quando usarlo: Quando vuoi solo l'idea generale prima dei dettagli.

Cosa controllare: Che la frase descriva il risultato, non i singoli passaggi.

33. Spiega questa riga specifica

"Non capisco questa riga: <riga>. Spiegamela da sola, in dettaglio, da principiante."

Quando usarlo: Quando un'unica riga ti blocca.

Cosa controllare: Che spieghi ogni simbolo/funzione presente nella riga.

34. Cosa stampa e perché

"Cosa stampa esattamente questo codice e perché? <codice>. Non eseguirlo solo: spiegami il ragionamento."

Quando usarlo: Quando vuoi prevedere l'output e verificare di aver capito.

Cosa controllare: Confronta la risposta con l'output reale eseguendolo in Colab.

35. Tradurre il codice in passi

"Trasforma questo codice in una lista di passi in italiano (pseudocodice), uno per riga di logica: <codice>."

Quando usarlo: Quando vuoi capire la logica senza la sintassi.

Cosa controllare: Che i passi rispecchino davvero cosa fa il codice.

36. Spiega la parte che non capisco

"Capisco la maggior parte di questo codice, tranne <parte specifica>. Spiegami solo quella parte: <codice>."

Quando usarlo: Quando ti manca solo un pezzo.

Cosa controllare: Che si concentri sulla parte indicata senza rispiegare tutto.

37. Cosa succede se cambio questo

"In questo codice, cosa succederebbe se cambiassi <X> con <Y>? <codice>. Spiegami l'effetto."

Quando usarlo: Quando vuoi capire un pezzo modificandolo mentalmente.

Cosa controllare: Verifica la previsione provando davvero la modifica.

38. Spiega i nomi delle variabili

"In questo codice, dimmi cosa rappresenta ogni variabile e se i nomi sono chiari: <codice>."

Quando usarlo: Quando i nomi delle variabili non ti aiutano a capire.

Cosa controllare: Che spieghi il ruolo di ogni variabile nel flusso del programma.

39. Spiega questo costrutto

"Cosa significa questa scrittura in Python: <costrutto, es. una list comprehension>? Spiegamela e mostrami la versione 'lunga' equivalente."

Quando usarlo: Quando incontri una scrittura compatta che non riconosci.

Cosa controllare: Che la versione lunga ti chiarisca cosa fa quella compatta.

40. Commenta il codice per me

"Aggiungi un commento chiaro a ogni riga di questo codice, per aiutarmi a studiarlo: <codice>."

Quando usarlo: Quando vuoi una versione annotata da rileggere.

Cosa controllare: Che i commenti spieghino il perché, non ripetano solo il cosa.

41. Spiega il flusso

"Spiegami in che ordine vengono eseguite le righe di questo codice, soprattutto dentro i cicli e gli if: <codice>."

Quando usarlo: Quando ci sono cicli e condizioni e ti perdi nel flusso.

Cosa controllare: Che chiarisca quante volte gira il ciclo e quando entra negli if.

42. Confronta due versioni

"Queste due versioni di codice fanno la stessa cosa? <codice A> e <codice B>. Quali differenze ci sono e quale è meglio per un principiante?"

Quando usarlo: Quando hai due soluzioni e non sai quale preferire.

Cosa controllare: Che spieghi i pro e contro, non solo dichiarare un vincitore.

Generare esercizi su misura

43. Esercizi facili su un argomento

"Creami 3 esercizi FACILI su <argomento> in Python, solo testo, senza soluzione, per un principiante assoluto. Li voglio risolvere da solo."

Quando usarlo: Quando vuoi allenarti su un argomento appena studiato.

Cosa controllare: Che siano davvero facili e mirati sull'argomento richiesto.

44. Esercizio con difficoltà crescente

"Dammi 3 esercizi su <argomento>: uno facile, uno medio, uno più impegnativo. Senza soluzioni."

Quando usarlo: Quando vuoi metterti alla prova progressivamente.

Cosa controllare: Che la difficoltà cresca davvero e resti alla portata di un principiante.

45. Esercizio sulla vita reale

"Creami un esercizio pratico su <argomento> ispirato a una situazione reale (spesa, voti, contatti...). Solo consegna, senza soluzione."

Quando usarlo: Quando vuoi esercizi che assomiglino a problemi veri.

Cosa controllare: Che la situazione sia concreta e risolvibile con ciò che sai.

46. Esercizio mirato sul mio punto debole

"Faccio fatica con <difficoltà specifica>. Creami 3 esercizi mirati proprio su quello, dal più semplice, senza soluzione."

Quando usarlo: Quando hai individuato una tua debolezza.

Cosa controllare: Che gli esercizi insistano sul punto debole, non su altro.

47. Esercizio 'trova l'errore'

"Creami 3 piccoli programmi Python su <argomento> che contengono un errore ciascuno. Io devo trovarlo. Non dirmi dov'è."

Quando usarlo: Per allenare l'occhio a riconoscere gli errori.

Cosa controllare: Che gli errori siano realistici e di tipo da principiante.

48. Esercizio di completamento

"Dammi un codice Python su <argomento> con alcune parti mancanti segnate con _____. Io devo completarle. Senza soluzione."

Quando usarlo: Quando vuoi un esercizio guidato, non da zero.

Cosa controllare: Che le parti mancanti siano significative, non banali.

49. Esercizio di lettura del codice

"Dammi un codice su <argomento> e chiedimi di prevedere l'output senza eseguirlo. Non darmi la risposta subito."

Quando usarlo: Per allenare la comprensione del codice.

Cosa controllare: Dopo aver risposto, esegui il codice per verificarti.

50. Quiz a risposta multipla

"Creami 5 domande a risposta multipla su <argomento> in Python per principianti, con 4 opzioni ciascuna. Le soluzioni mettile in fondo, separate."

Quando usarlo: Per un ripasso veloce e misurabile.

Cosa controllare: Che ci sia una sola risposta chiaramente corretta per domanda.

51. Set di esercizi per una settimana

"Preparami un piano di 5 esercizi (uno al giorno) su <argomento>, di difficoltà crescente, senza soluzioni."

Quando usarlo: Quando vuoi un programma di allenamento.

Cosa controllare: Che il carico giornaliero sia sostenibile per un principiante.

52. Esercizio che combina due argomenti

"Creami un esercizio che metta insieme <argomento A> e <argomento B>, per principiante, senza soluzione."

Quando usarlo: Quando vuoi collegare concetti già studiati separatamente.

Cosa controllare: Che entrambi gli argomenti servano davvero per risolverlo.

53. Esercizio capstone

"Creami un mini-progetto finale che usi <elenco di argomenti>. Dammi solo la traccia dettagliata, senza codice."

Quando usarlo: Quando vuoi un progetto che riassume più argomenti.

Cosa controllare: Che la traccia sia chiara e divisibile in passi piccoli.

54. Varianti dello stesso esercizio

“Dammi 3 varianti di questo esercizio cambiando solo il contesto, per allenarmi sullo stesso concetto: <esercizio>.”

Quando usarlo: Quando vuoi consolidare un concetto con la ripetizione.

Cosa controllare: Che il concetto allenato resti lo stesso in tutte le varianti.

55. Esercizio con dati da pulire

“Creami un esercizio in cui ricevo dati 'sporchi' (spazi, maiuscole, virgole) da pulire con Python base. Senza soluzione.”

Quando usarlo: Per allenarti sulla pulizia dei dati.

Cosa controllare: Che i dati sporchi siano realistici e gestibili con ciò che sai.

56. Esercizio sui dizionari realistico

“Creami un esercizio dove uso un dizionario per rappresentare <cosa, es. un prodotto>. Solo consegna.”

Quando usarlo: Per praticare i dizionari su casi concreti.

Cosa controllare: Che richieda di accedere e modificare il dizionario, non solo crearlo.

57. Esercizio con file

“Creami un esercizio in cui scrivo e poi rileggo un file di testo in Python. Solo consegna, senza soluzione.”

Quando usarlo: Per praticare la lettura/scrittura di file.

Cosa controllare: Che includa sia la scrittura sia la riletture del file.

Rivedere e migliorare il tuo codice

58. Revisione da principiante

“Ho scritto questo codice da principiante: <codice>. Funziona, ma è scritto bene? Dimmi cosa migliorare, spiegando ogni suggerimento in modo semplice.”

Quando usarlo: Dopo aver risolto un esercizio, per migliorare.

Cosa controllare: Che i suggerimenti siano alla tua portata e spiegati, non solo elencati.

59. Rendere il codice più leggibile

“Come posso rendere questo codice più chiaro e leggibile, senza renderlo più complicato? <codice>”

Quando usarlo: Quando il codice funziona ma ti sembra confuso.

Cosa controllare: Che non introduca concetti avanzati che non conosci.

60. Nomi di variabili migliori

“Suggeriscimi nomi di variabili più chiari per questo codice, spiegando perché: <codice>.”

Quando usarlo: Quando usi nomi tipo x, a, b poco comprensibili.

Cosa controllare: Che i nuovi nomi rendano il codice auto-esplicativo.

61. Trovare ripetizioni

“In questo codice ci sono parti ripetute che potrei semplificare? <codice>. Mostrami come, da principiante.”

Quando usarlo: Quando ti accorgi di copiare/incollare codice simile.

Cosa controllare: Che la semplificazione resti comprensibile (es. una funzione o un ciclo).

62. Aggiungere i commenti giusti

“Dove dovrei aggiungere commenti in questo codice e cosa dovrebbero dire? <codice>”

Quando usarlo: Quando vuoi imparare a commentare bene.

Cosa controllare: Che suggerisca commenti sul 'perché', non sul 'cosa' ovvio.

63. Trasformare in funzione

“Questo pezzo di codice lo uso più volte: <codice>. Come lo trasformo in una funzione riutilizzabile? Spiegami i passaggi.”

Quando usarlo: Quando ripeti lo stesso blocco in punti diversi.

Cosa controllare: Che la funzione abbia parametri e return chiari e tu capisca come chiamarla.

64. Gestire i casi limite

“Questo codice funziona coi dati normali: <codice>. Quali casi limite (lista vuota, zero, testo invece di numero) potrebbero romperlo?”

Quando usarlo: Quando vuoi rendere il codice più robusto.

Cosa controllare: Che i casi limite siano realistici; prova a gestirne almeno uno.

65. Confronto con la soluzione 'ideale'

“Ho risolto così: <tuoi codice>. Mostrami come l'avrebbe scritto un programmatore esperto, ma spiegandomi ogni differenza da principiante.”

Quando usarlo: Dopo aver risolto, per imparare uno stile migliore.

Cosa controllare: Che spieghi le differenze; non copiare ciecamente la versione 'esperta'.

66. Codice più corto ma chiaro

“Posso scrivere questo codice in modo più breve SENZA renderlo più difficile da capire? <codice>”

Quando usarlo: Quando il codice ti sembra inutilmente lungo.

Cosa controllare: Che la versione corta non usi trucchi che non capisci.

67. Spiega i miei stessi errori di stile

“Quali cattive abitudini di principiante vedi in questo mio codice? <codice>. Così le correggo.”

Quando usarlo: Per crescere come programmatore, non solo far funzionare il codice.

Cosa controllare: Che le abitudini segnalate siano spiegate con l'alternativa giusta.

68. Validare gli input dell'utente

“Questo codice chiede dati all'utente: <codice>. Come controllo che l'utente abbia inserito qualcosa di valido?”

Quando usarlo: Quando il tuo programma riceve input dalle persone.

Cosa controllare: Che proponga controlli semplici (es. try/except, if) comprensibili.

69. Formattare bene l'output

“Come posso rendere l'output di questo codice più ordinato e leggibile? <codice>”

Quando usarlo: Quando i risultati a schermo sono disordinati.

Cosa controllare: Che usi strumenti che conosci (f-string, .2f, allineamenti).

70. Capire se ho 'barato'

“Ho risolto questo esercizio con l'aiuto dell'AI: <codice>. Voglio essere sicuro di averlo capito: interrogami con 3 domande sul mio stesso codice.”

Quando usarlo: Dopo aver usato l'AI, per verificare di non aver copiato a vuoto.

Cosa controllare: Se non sai rispondere alle domande, rivedi il codice prima di proseguire.

71. Rendere il codice più sicuro

“Questo codice può bloccarsi con dati strani: <codice>. Come lo proteggerò con try/except, da principiante?”

Quando usarlo: Quando il programma deve gestire input imprevedibili.

Cosa controllare: Che catturi errori specifici e tu capisca cosa fa ogni except.

72. Spiegami il mio codice come se non fosse mio

“Spiegami questo codice come se non sapessi chi l'ha scritto, evidenziando cosa non è chiaro: <codice>.”

Quando usarlo: Per vedere il tuo codice con occhi nuovi.

Cosa controllare: Che evidenzi i punti poco chiari: sono quelli da migliorare.

Tradurre un'idea in codice

73. Da problema a passi

“Voglio scrivere un programma che <obiettivo>. Non darmi il codice: aiutami a scomporlo in passi semplici che posso programmare uno alla volta.”

Quando usarlo: Quando hai un'idea ma non sai da dove iniziare.

Cosa controllare: Che i passi siano piccoli e ordinati; prova a scriverne il codice da solo.

74. Da passi a codice (guidato)

“Ho diviso il mio problema in questi passi: <passi>. Aiutami a scrivere il PRIMO passo in Python, spiegandolo, poi mi fermo e provo gli altri.”

Quando usarlo: Quando hai i passi ma ti blocchi a scrivere il codice.

Cosa controllare: Che spieghi il primo passo; scrivi tu i successivi prima di chiedere ancora.

75. Quale strumento mi serve

“Per fare <obiettivo> in Python, quale strumento mi conviene usare (lista, dizionario, ciclo, funzione...)? Spiegami perché.”

Quando usarlo: Quando non sai quale costrutto usare per un problema.

Cosa controllare: Che motivi la scelta e la colleghi al tuo obiettivo.

76. Bozza da completare io

“Dammi lo scheletro (con commenti al posto del codice) di un programma che <obiettivo>, così lo completo io.”

Quando usarlo: Quando vuoi una guida ma non la soluzione pronta.

Cosa controllare: Che lo scheletro lasci a te le parti di logica importanti.

77. Pseudocodice prima del codice

“Scrivimi in pseudocodice (italiano, non Python) come risolvere <obiettivo>, così poi lo traduco io in Python.”

Quando usarlo: Quando vuoi ragionare sulla logica prima della sintassi.

Cosa controllare: Che lo pseudocodice sia chiaro e traducibile passo per passo.

78. Esempio simile da adattare

“Mostrami un esempio semplice di un programma che fa qualcosa di SIMILE a <obiettivo>, così capisco l'approccio e lo adatto.”

Quando usarlo: Quando vuoi un modello da cui partire.

Cosa controllare: Che l'esempio sia adattabile e tu capisca cosa cambiare.

79. Validare la mia idea

“Voglio risolvere <problema> così: <tua idea>. È un buon approccio per un principiante? Ci sono modi più semplici?”

Quando usarlo: Quando hai un'idea ma vuoi un parere prima di scriverla.

Cosa controllare: Che valuti la tua idea e proponga alternative solo se più semplici.

80. Da Excel a Python

“Faccio questo a mano/in Excel: <descrizione>. Come imposterei la stessa cosa in Python, concettualmente? Niente codice complicato.”

Quando usarlo: Quando vuoi automatizzare qualcosa che fai a mano.

Cosa controllare: Che colleghi i passi manuali a costrutti Python che conosci.

81. Scegliere tra due approcci

“Per fare <obiettivo> potrei usare <approccio A> o <approccio B>. Quale è più adatto a un principiante e perché?”

Quando usarlo: Quando hai due strade e non sai quale prendere.

Cosa controllare: Che la scelta sia motivata in base alla semplicità per te.

82. Stimare la difficoltà

“Quanto è difficile, per un principiante, scrivere un programma che <obiettivo>? Quali conoscenze mi servono prima di provarci?”

Quando usarlo: Prima di iniziare un progetto, per capire se sei pronto.

Cosa controllare: Che elenchi i prerequisiti; se ti mancano, studiali prima.

83. Dividere un progetto grande

“Voglio fare <progetto>. Aiutami a dividerlo in 4-5 mini-obiettivi che posso completare uno alla volta.”

Quando usarlo: Quando un progetto sembra troppo grande.

Cosa controllare: Che ogni mini-obiettivo sia completabile da solo e in poco tempo.

84. Domande da farmi prima di programmare

“Prima di scrivere un programma che <obiettivo>, quali domande dovrei pormi su input, elaborazione e output?”

Quando usarlo: Per imparare a impostare bene un problema.

Cosa controllare: Che le domande chiariscano cosa entra, cosa succede e cosa esce.

85. Tradurre una formula in codice

“Ho questa formula/regola: <formula a parole>. Come la scrivo in Python? Spiegami la traduzione.”

Quando usarlo: Quando devi trasformare un calcolo in codice.

Cosa controllare: Che la traduzione rispetti la formula; verificala con un esempio numerico.

86. Capire i requisiti

“Mi hanno chiesto di fare <richiesta vaga>. Aiutami a trasformarla in requisiti chiari e specifici prima di programmare.”

Quando usarlo: Quando la richiesta è generica e non sai cosa fare di preciso.

Cosa controllare: Che i requisiti siano concreti e misurabili.

87. Prototipo minimo

“Qual è la versione più semplice possibile di <obiettivo> che potrei far funzionare per prima, prima di aggiungere il resto?”

Quando usarlo: Quando vuoi partire da qualcosa che già funziona.

Cosa controllare: Che il prototipo sia davvero minimo e funzionante.

Debug: trovare i bug

88. Aggiungere print per capire

“Il mio codice non fa quello che voglio: <codice>. Dimmi dove aggiungere delle print() per capire cosa succede passo passo.”

Quando usarlo: Quando non capisci perché il risultato è sbagliato.

Cosa controllare: Che ti faccia 'vedere' i valori nei punti chiave; poi rimuovi le print.

89. Spiega cosa vale ogni variabile

“Esegui mentalmente questo codice e dimmi quanto vale ogni variabile a ogni passo: <codice>.”

Quando usarlo: Quando ti perdi nel flusso dei valori.

Cosa controllare: Verifica con print() reali in Colab.

90. Isolare il problema

“Questo programma lungo ha un bug: <codice>. Aiutami a capire in quale parte si trova, dividendolo in blocchi.”

Quando usarlo: Quando il codice è lungo e non sai dove guardare.

Cosa controllare: Che restringa il campo a un blocco specifico.

91. Bug logico vs bug di sintassi

“Il mio codice non dà errori ma sbaglia il risultato. È un problema di logica? <codice>. Spiegami.”

Quando usarlo: Quando non c'è messaggio rosso ma il risultato è errato.

Cosa controllare: Che distingua tra errore di scrittura ed errore di ragionamento.

92. Perché il ciclo non si comporta così

“Mi aspettavo che questo ciclo facesse <atteso>, invece fa <ottenuto>: <codice>. Perché?”

Quando usarlo: Quando un for/while non si comporta come pensavi.

Cosa controllare: Che spieghi quante volte gira e cosa cambia a ogni giro.

93. Condizione che non scatta mai

“Questo if non viene mai eseguito (o viene sempre eseguito): <codice>. Perché?”

Quando usarlo: Quando una condizione si comporta in modo strano.

Cosa controllare: Che controlli i tipi e gli operatori (== vs =, and/or).

94. Debug di un input

“Quando inserisco <valore> il programma sbaglia: <codice>. Cosa succede con quell'input?”

Quando usarlo: Quando il bug appare solo con certi dati.

Cosa controllare: Che spieghi come quell'input attraversa il codice.

95. Metodo di debug per principianti

“Insegnami un metodo semplice e generale per trovare i bug nel mio codice, passo per passo.”

Quando usarlo: Quando vuoi imparare a fare debug da solo.

Cosa controllare: Che il metodo sia ripetibile e non legato a un caso singolo.

96. Verifica un pezzo alla volta

“Come faccio a testare questo codice un pezzo alla volta invece che tutto insieme? <codice>”

Quando usarlo: Quando il programma è complesso.

Cosa controllare: Che suggerisca di provare le funzioni/blocchi separatamente.

97. Capire un risultato inatteso

“Perché questo calcolo dà <risultato strano, es. 0.30000000000000004>? <codice>”

Quando usarlo: Quando i numeri decimali danno risultati bizzarri.

Cosa controllare: Che spieghi le imprecisioni dei float e l'uso di round().

98. Bug che appare solo a volte

“Il programma a volte funziona e a volte no: <codice>. Da cosa può dipendere?”

Quando usarlo: Quando il comportamento è incostante.

Cosa controllare: Che consideri input diversi o l'ordine delle celle in Colab.

99. Leggi il mio ragionamento

“Ti spiego cosa pensavo facesse il codice: <ragionamento>. Dove il codice si comporta diversamente da come pensavo? <codice>”

Quando usarlo: Quando il tuo modello mentale non torna col risultato.

Cosa controllare: Che individui il punto in cui la tua idea e il codice divergono.

Lavorare con i dati e i CSV

100. Capire un file CSV

“Ho un file CSV con queste colonne: <colonne>. Spiegami come è strutturato e come lo leggo in Python base.”

Quando usarlo: Quando inizi a lavorare con dati veri.

Cosa controllare: Che spieghi righe, separatori e intestazione.

101. Leggere un CSV riga per riga

“Come leggo questo CSV riga per riga e separo i campi, in Python base (senza librerie)? Colonne: <colonne>.”

Quando usarlo: Quando vuoi elaborare un CSV con le basi.

Cosa controllare: Che usi split() e converta i campi numerici con int()/float().

102. Trasformare righe in dizionari

“Come trasformo ogni riga di un CSV in un dizionario usando le intestazioni come chiavi? Colonne: <colonne>.”

Quando usarlo: Quando vuoi dati più comodi da usare.

Cosa controllare: Che il risultato sia una lista di dizionari leggibile.

103. Calcolare un totale da un CSV

“Ho un CSV con una colonna <colonna numerica>. Come calcolo il totale in Python base? Spiegami i passaggi.”

Quando usarlo: Quando devi sommare una colonna di dati.

Cosa controllare: Che ricordi di saltare l'intestazione e convertire in numero.

104. Filtrare righe per criterio

“Come tengo solo le righe del mio CSV dove <colonna> è <condizione>? Spiegamelo da principiante.”

Quando usarlo: Quando vuoi selezionare un sottoinsieme di dati.

Cosa controllare: Che usi un ciclo con un if comprensibile.

105. Trovare il massimo nei dati

“Come trovo la riga con il valore più alto nella colonna <colonna> del mio CSV?”

Quando usarlo: Quando cerchi il record 'top' nei tuoi dati.

Cosa controllare: Che spieghi come confrontare e tenere il massimo.

106. Raggruppare e sommare

“Come raggruppo i miei dati per <colonna categoria> e sommo <colonna valore>, in Python base? Spiegami con un dizionario.”

Quando usarlo: Quando vuoi totali per categoria.

Cosa controllare: Che usi un dizionario come registro dei totali.

107. Pulire una colonna

“La colonna <colonna> del mio CSV ha valori sporchi (spazi, maiuscole, virgole). Come la pulisco in Python base?”

Quando usarlo: Quando i dati reali sono disordinati.

Cosa controllare: Che usi strip(), lower(), replace() in modo comprensibile.

108. Gestire righe rotte

“Il mio CSV ha qualche riga rotta che fa crashare il programma. Come le salto senza fermarmi? <codice>”

Quando usarlo: Quando i dati reali contengono errori.

Cosa controllare: Che usi try/except e continui con le righe valide.

109. Scrivere i risultati in un file

“Come salvo i risultati della mia analisi in un nuovo file di testo/CSV, in Python base?”

Quando usarlo: Quando vuoi conservare l'output.

Cosa controllare: Che ricordi encoding utf-8 e l'a-capo per ogni riga.

110. Convertire numeri italiani

“Il mio CSV ha prezzi come '1.234,50' (formato italiano). Come li converto in numeri usabili in Python?”

Quando usarlo: Quando lavori con dati in formato italiano.

Cosa controllare: Che gestisca virgola decimale e separatore delle migliaia.

111. Statistiche di base sui dati

“Ho una lista di numeri dai miei dati: <descrizione>. Come calcolo conteggio, somma, media, minimo e massimo?”

Quando usarlo: Quando vuoi un primo riassunto statistico.

Cosa controllare: Che usi len(), sum(), min(), max() correttamente.

112. Capire i miei dati

“Ho questi dati: <descrizione/esempio>. Aiutami a capire quali domande utili potrei rispondere con Python base.”

Quando usarlo: Quando hai dei dati ma non sai cosa farci.

Cosa controllare: Che proponga analisi alla tua portata.

113. Da dati a mini-report

“Come trasformo questa analisi in un report leggibile e ordinato a schermo? <descrizione>”

Quando usarlo: Quando vuoi presentare i risultati bene.

Cosa controllare: Che usi f-string e formattazione che conosci.

114. Verificare la qualità dei dati

“Quali controlli semplici posso fare per scoprire se i miei dati hanno problemi (vuoti, doppi, valori strani)?”

Quando usarlo: Prima di analizzare, per fidarti dei dati.

Cosa controllare: Che proponga controlli concreti con ciò che sai.

Mini-progetti guidati

115. Idea di progetto adatta a me

“Suggeriscimi 3 idee di mini-progetto Python adatte a un principiante che ha studiato <argomenti>. Solo idee, non codice.”

Quando usarlo: Quando vuoi mettere in pratica ciò che sai.

Cosa controllare: Che le idee siano realizzabili con le tue conoscenze attuali.

116. Traccia di un progetto

“Voglio costruire <progetto, es. un gestore di spese>. Dammi una traccia dettagliata divisa in passi, senza codice.”

Quando usarlo: Quando hai un'idea di progetto ma non un piano.

Cosa controllare: Che i passi siano ordinati e ognuno completabile da solo.

117. Funzionalità minime

“Per un <progetto>, quali sono le 3 funzionalità minime da cui partire prima di aggiungere il resto?”

Quando usarlo: Quando vuoi evitare di partire troppo in grande.

Cosa controllare: Che le funzionalità minime siano davvero essenziali.

118. Struttura dati del progetto

“Per il mio <progetto>, come conviene organizzare i dati (liste, dizionari, file)? Spiegami la scelta.”

Quando usarlo: All'inizio di un progetto, per impostare bene i dati.

Cosa controllare: Che la struttura proposta sia semplice e adatta al caso.

119. Menu a scelte

“Voglio un programma con un menu (1 aggiungi, 2 mostra, 3 esci...). Spiegami lo schema generale con un ciclo while, senza farmi tutto il codice.”

Quando usarlo: Quando vuoi un programma interattivo a menu.

Cosa controllare: Che spieghi il while e gli if delle scelte; il codice scrivilo tu.

120. Salvare i dati del progetto

“Nel mio <progetto> voglio che i dati restino salvati anche dopo la chiusura. Come faccio, in Python base, concettualmente?”

Quando usarlo: Quando vuoi persistenza dei dati.

Cosa controllare: Che proponga la scrittura su file in modo comprensibile.

121. Aggiungere una funzione al progetto

“Il mio <progetto> fa già <X>. Voglio aggiungere <Y>. Aiutami a capire dove e come inserirlo, senza riscrivere tutto.”

Quando usarlo: Quando estendi un progetto esistente.

Cosa controllare: Che indichi il punto giusto senza stravolgere il codice.

122. Capstone AI Ready

“Aiutami a impostare un progetto finale che usi dati, un po' di automazione e l'AI come supporto. Dammi solo la traccia.”

Quando usarlo: Quando vuoi un progetto che riassume il percorso.

Cosa controllare: Che la traccia integri i temi senza superare il tuo livello.

123. Testare il mio progetto

“Come provo che il mio <progetto> funzioni davvero in tutti i casi? Dammi una lista di prove da fare.”

Quando usarlo: Quando hai finito e vuoi verificarlo.

Cosa controllare: Che le prove includano casi normali e casi limite.

124. Migliorare un progetto finito

“Ho finito <progetto>: <codice o descrizione>. Quali 3 miglioramenti semplici potrei aggiungere?”

Quando usarlo: Quando vuoi far crescere un progetto che funziona.

Cosa controllare: Che i miglioramenti siano alla tua portata.

125. Progetto a tema lavoro

“Suggeriscimi un mini-progetto Python utile nel mio lavoro/ambito: <ambito>. Solo idea e traccia.”

Quando usarlo: Quando vuoi un progetto utile nella tua realtà.

Cosa controllare: Che sia concreto e realizzabile con Python base.

126. Spiega l'architettura a parole

“Spiegami a parole come dovrebbero essere organizzate le parti del mio <progetto> (dati, funzioni, menu), senza codice.”

Quando usarlo: Quando vuoi capire la struttura prima di scrivere.

Cosa controllare: Che la descrizione sia chiara e traducibile in codice.

Ripasso e autoverifica

127. Ripasso veloce di un argomento

“Fammi un ripasso veloce di <argomento> in Python in 5 punti, per principianti.”

Quando usarlo: Prima di un esercizio o dopo una pausa dallo studio.

Cosa controllare: Che i 5 punti coprano l'essenziale dell'argomento.

128. Interrogami su un argomento

“Interrogami su <argomento> facendomi una domanda alla volta e aspettando la mia risposta prima della successiva.”

Quando usarlo: Quando vuoi un ripasso attivo, non passivo.

Cosa controllare: Che dia un feedback sulla tua risposta prima di proseguire.

129. Carte da ripasso

“Creami 8 'flashcard' (domanda/risposta) su <argomento> in Python per principianti.”

Quando usarlo: Per memorizzare concetti chiave.

Cosa controllare: Che le domande siano brevi e le risposte precise.

130. Spiegami cosa NON ho capito

"Ti spiego <argomento> con parole mie: <spiegazione>. Dimmi cosa non è chiaro o sbagliato nel mio modo di spiegarlo."

Quando usarlo: Dopo lo studio, per scoprire i buchi.

Cosa controllare: Che evidenzi gli errori in modo specifico.

131. Mini-test a tempo

"Creami un mini-test di 5 domande su <argomento> che posso fare in 10 minuti. Risposte in fondo."

Quando usarlo: Quando vuoi misurarti rapidamente.

Cosa controllare: Che il test sia svolgibile nel tempo indicato.

132. Cosa ripassare prima di proseguire

"Sto per studiare <argomento nuovo>. Cosa dovrei già sapere bene prima di iniziare?"

Quando usarlo: Prima di affrontare un argomento nuovo.

Cosa controllare: Che elenchi prerequisiti chiari; se ti mancano, ripassali.

133. Autovalutazione onesta

"Fammi 5 domande per capire se sono pronto a considerarmi a mio agio con <argomento>."

Quando usarlo: Quando vuoi sapere se hai consolidato un argomento.

Cosa controllare: Che le domande tocchino i punti centrali, non i dettagli.

134. Collega gli argomenti

"Aiutami a vedere come <argomento A> e <argomento B> si collegano tra loro in Python."

Quando usarlo: Quando hai studiato cose separate e vuoi unirle.

Cosa controllare: Che il collegamento sia corretto e utile.

135. Riassunto dell'intero libro

"Riassumi in 10 punti le competenze chiave che dovrei avere dopo un corso base di Python."

Quando usarlo: A fine percorso, per fare il punto.

Cosa controllare: Che i punti rispecchino le basi (variabili, cicli, funzioni, dati...).

136. Dove sono debole

"In base a queste mie risposte/errori <descrizione>, su quali argomenti dovrei concentrarmi di più?"

Quando usarlo: Quando vuoi capire dove investire lo studio.

Cosa controllare: Che indichi argomenti specifici, non consigli generici.

137. Esercizio di ripasso misto

"Creami un esercizio che mescola più argomenti che ho già studiato (<elenco>), per ripassare tutto insieme."

Quando usarlo: Per un ripasso generale prima di andare avanti.

Cosa controllare: Che richiami davvero più argomenti dell'elenco.

138. Spiega la differenza dopo l'errore

"Ho sbagliato perché confondevo <X> e <Y>. Spiegamela differenza in modo che non li confonda mai più."

Quando usarlo: Dopo un errore dovuto a confusione tra concetti.

Cosa controllare: Che dia un trucco per ricordare la differenza.

139. Verifica con un esempio mio

"Ho capito <concetto>? Ti faccio un esempio mio: <esempio>. Dimmi se l'ho usato correttamente."

Quando usarlo: Per verificare la comprensione con un caso tuo.

Cosa controllare: Che valuti il tuo esempio specifico, non in generale.

140. Domande sempre più difficili

"Interrogami su <argomento> partendo facile e aumentando la difficoltà a ogni risposta corretta."

Quando usarlo: Quando vuoi spingerti gradualmente.

Cosa controllare: Che aumenti davvero la difficoltà e si fermi dove arrivi.

141. Ripasso prima di dormire

"Dammi 3 concetti chiave di <argomento> da rileggere stasera, spiegati in modo brevissimo."

Quando usarlo: Per un ripasso leggero e veloce.

Cosa controllare: Che siano i concetti più importanti, in poche righe.

Metodo di studio e motivazione

142. Piano di studio settimanale

“Aiutami a creare un piano di studio di Python di <ore> a settimana per <settimane>, da principiante. Realistico e sostenibile.”

Quando usarlo: Quando vuoi organizzare lo studio.

Cosa controllare: Che il piano sia realistico e non troppo carico.

143. Cosa fare quando mi blocco

“Mi blocco spesso e mi scoraggio. Dammi una strategia in 4 passi da seguire quando non riesco a risolvere un esercizio.”

Quando usarlo: Quando la frustrazione ti ferma.

Cosa controllare: Che i passi includano provare da solo prima di chiedere aiuto.

144. Studiare poco ma bene

“Ho poco tempo. Come studio Python in modo efficace anche con sessioni brevi?”

Quando usarlo: Quando hai poco tempo a disposizione.

Cosa controllare: Che proponga abitudini sostenibili, non scorciatoie illusorie.

145. Non perdere la motivazione

“Dammi 3 consigli pratici per non mollare quando imparare a programmare diventa difficile.”

Quando usarlo: Nei momenti di sconforto.

Cosa controllare: Che i consigli siano concreti e applicabili subito.

146. Imparare dagli errori

“Come trasformo i miei errori di programmazione in occasioni per imparare, invece di frustrarmi?”

Quando usarlo: Quando gli errori ti demoralizzano.

Cosa controllare: Che proponga un atteggiamento pratico verso l'errore.

147. Ritmo giusto

“Sto andando troppo veloce o troppo piano? Studio <ritmo attuale>. Come capisco il ritmo giusto per me?”

Quando usarlo: Quando dubiti del tuo passo di studio.

Cosa controllare: Che dia segnali concreti per capire se consolidi davvero.

148. Ripasso a distanza

“Spiegami come usare il ripasso distanziato nel tempo per non dimenticare quello che studio in Python.”

Quando usarlo: Quando dimentichi le cose dopo pochi giorni.

Cosa controllare: Che proponga un semplice schema di ripasso ripetuto.

149. Studiare con l'AI senza dipendere

“Come uso l'AI per studiare Python senza diventarne dipendente e continuando a ragionare con la mia testa?”

Quando usarlo: Quando temi di affidarti troppo all'AI.

Cosa controllare: Che ribadisca: prima provi tu, poi chiedi.

150. Costruire l'abitudine

“Aiutami a trasformare lo studio di Python in un'abitudine quotidiana, con un piccolo gesto da fare ogni giorno.”

Quando usarlo: Quando fatichi a essere costante.

Cosa controllare: Che proponga un'abitudine piccola e ripetibile.

151. Festeggiare i progressi

“Come tengo traccia dei miei progressi in Python per restare motivato?”

Quando usarlo: Quando ti sembra di non migliorare.

Cosa controllare: Che proponga un modo concreto di misurare i progressi (es. il registro del kit).

152. Superare la paura del codice

“Ho paura di sbagliare e di rompere qualcosa quando scrivo codice. Come supero questa paura, da principiante?”

Quando usarlo: Quando l'ansia ti blocca prima ancora di provare.

Cosa controllare: Che rassicuri in modo realistico: in Colab non rompi nulla di grave.

153. Imparare facendo

“Sto solo leggendo senza scrivere codice. Come passo a imparare facendo, in modo pratico?”

Quando usarlo: Quando studi in modo troppo passivo.

Cosa controllare: Che spinga a eseguire e modificare il codice, non solo leggerlo.

Usare l'AI in modo critico e sicuro

154. Verificare una risposta dell'AI

“Mi hai dato questo codice: <codice>. Spiegamelo riga per riga così verifico di averlo capito prima di usarlo.”

Quando usarlo: Sempre, dopo aver ricevuto codice dall'AI.

Cosa controllare: Che tu capisca ogni riga; se no, non usarlo ancora.

155. Chiedere alternative più semplici

“Questa soluzione usa cose che non conosco: <codice>. Me ne dai una versione più semplice, con solo le basi?”

Quando usarlo: Quando l'AI usa concetti troppo avanzati.

Cosa controllare: Che la nuova versione usi solo ciò che hai già studiato.

156. Mettere in dubbio l'AI

“Sei sicuro di questa risposta? Spiegami perché è corretta e mostrami come potrei verificarla da solo.”

Quando usarlo: Quando una risposta ti convince poco.

Cosa controllare: Che fornisca un modo per verificare in autonomia.

157. Capire i limiti

“Su cosa l'AI potrebbe sbagliare riguardo a <argomento> Python? Cosa dovrei verificare sempre?”

Quando usarlo: Quando vuoi sapere di cosa diffidare.

Cosa controllare: Che indichi punti specifici da controllare.

158. Niente codice, solo guida

“Non darmi il codice della soluzione. Aiutami solo con indizi e domande, così ci arrivo da solo: <problema>.”

Quando usarlo: Quando vuoi imparare davvero, non copiare.

Cosa controllare: Che resista alla tentazione di darti la soluzione pronta.

159. Spiega senza fare al posto mio

“Spiegami il concetto che mi serve per risolvere <problema>, ma lascia che sia io a scrivere il codice.”

Quando usarlo: Quando vuoi capire ma fare da solo.

Cosa controllare: Che spieghi il concetto senza risolvere l'esercizio.

160. Confrontare due risposte AI

“Ho ricevuto due risposte diverse alla stessa domanda: <A> e . Quale è corretta e perché?”

Quando usarlo: Quando ottieni risposte contrastanti.

Cosa controllare: Che motivi la scelta e indichi come verificarla.

161. Adattare al mio livello

“D'ora in poi, in questa conversazione, rispondimi sempre da principiante assoluto di Python, con parole semplici ed esempi.”

Quando usarlo: All'inizio di una sessione di studio con l'AI.

Cosa controllare: Che mantenga il livello semplice; ricordaglielo se torna tecnico.

162. Chiedere il perché, non solo il come

“Non dirmi solo come si fa <cosa>, dimmi anche perché si fa così.”

Quando usarlo: Quando vuoi capire, non solo eseguire.

Cosa controllare: Che la spiegazione del 'perché' sia chiara.

163. Trappola della sicurezza

“Questo codice sembra giusto ma non sono sicuro: <codice>. Aiutami a metterlo alla prova con un esempio che potrebbe smascherare un errore.”

Quando usarlo: Quando un codice 'sembra' funzionare.

Cosa controllare: Che proponga un caso di prova significativo.

164. Imparare a chiedere meglio

"Ho chiesto <prompt> e ho avuto una risposta poco utile. Come avrei dovuto formulare meglio la domanda?"

Quando usarlo: Quando una risposta non ti soddisfa.

Cosa controllare: Che ti insegni a rendere il prompt più specifico.

165. Riconoscere il copia-incolla inutile

"Ho copiato questo codice senza capirlo: <codice>. Aiutami a capirlo ora, prima di andare avanti."

Quando usarlo: Quando ti accorgi di aver copiato senza capire.

Cosa controllare: Che ti porti a comprenderlo davvero, non a ricopiarlo.

Google Colab e ambiente

166. Capire Colab

"Spiegami in modo semplice cos'è Google Colab e come funzionano le celle, da principiante assoluto."

Quando usarlo: All'inizio, per capire lo strumento.

Cosa controllare: Che spieghi celle, esecuzione e ordine in modo chiaro.

167. Errore 'name not defined' in Colab

"In Colab ottengo 'name not defined' su una variabile che ho scritto in un'altra cella. Perché? Come risolvo?"

Quando usarlo: Quando le celle non 'comunicano' come pensavi.

Cosa controllare: Che spieghi l'importanza dell'ordine di esecuzione.

168. Riavviare il runtime

"Quando e perché dovrei riavviare il runtime in Colab? Spiegamelo da principiante."

Quando usarlo: Quando Colab fa cose strane.

Cosa controllare: Che spieghi cosa succede ai dati/variabili al riavvio.

169. Salvare il lavoro in Colab

"Come salvo il mio notebook Colab e dove finisce? Voglio essere sicuro di non perderlo."

Quando usarlo: Quando temi di perdere il lavoro.

Cosa controllare: Che spieghi il salvataggio su Google Drive.

170. Caricare un file in Colab

"Come carico un file (es. un CSV) in Colab per usarlo nel mio codice, da principiante?"

Quando usarlo: Quando devi lavorare con un file tuo.

Cosa controllare: Che indichi un metodo semplice e i possibili intoppi.

171. Scorciatoie utili

"Dammi 5 scorciatoie da tastiera utili in Google Colab per un principiante."

Quando usarlo: Per lavorare più velocemente.

Cosa controllare: Che siano scorciatoie davvero usate e funzionanti.

172. Ordine delle celle

"Spiegami perché in Colab l'ordine in cui eseguo le celle è importante e come evitare confusione."

Quando usarlo: Quando ottieni risultati incoerenti.

Cosa controllare: Che suggerisca di eseguire dall'alto verso il basso.

173. Pulire l'output

"Come pulisco l'output di una cella o azzerò tutto per ripartire da capo in Colab?"

Quando usarlo: Quando l'ambiente è 'sporco' da prove precedenti.

Cosa controllare: Che spieghi la differenza tra pulire output e riavviare runtime.

174. Installare una libreria

"Come installo una libreria in Colab quando un esercizio la richiede? Spiegamelo semplice."

Quando usarlo: Quando un esercizio chiede una libreria non base.

Cosa controllare: Che mostri il comando e quando serve.

175. Condividere un notebook

"Come condivido il mio notebook Colab con qualcun altro?"

Quando usarlo: Quando vuoi mostrare il tuo lavoro.

Cosa controllare: Che spieghi le opzioni di condivisione e i permessi.

Verso il passo successivo (dati, automazione, AI)

176. Cosa studiare dopo le basi

“Ho imparato le basi di Python (variabili, cicli, funzioni, liste, dizionari). Cosa mi conviene studiare dopo se mi interessano i dati?”

Quando usarlo: Quando hai finito le basi e vuoi continuare.

Cosa controllare: Che proponga un percorso graduale e realistico.

177. Introduzione gentile a pandas

“Spiegami a grandi linee cos'è pandas e perché è utile per i dati, senza ancora insegnarmelo. Solo l'idea.”

Quando usarlo: Quando senti parlare di pandas e vuoi capire cos'è.

Cosa controllare: Che resti a livello di idea, senza sovraccaricarti.

178. Primo passo verso l'automazione

“Voglio automatizzare <compito ripetitivo>. È qualcosa che posso fare con le basi di Python? Da dove parto?”

Quando usarlo: Quando hai un compito noioso da automatizzare.

Cosa controllare: Che valuti se è alla tua portata e indichi il primo passo.

179. Capire l'AI generativa

“Spiegami in modo semplice come funziona, a grandi linee, un'AI generativa come quella che uso per studiare.”

Quando usarlo: Quando sei curioso di capire lo strumento che usi.

Cosa controllare: Che resti semplice e onesto sui limiti.

180. Python per il reporting

“Come potrei usare Python per creare report o analisi nel mio lavoro (<ambito>)? Solo idee, livello base.”

Quando usarlo: Quando vuoi applicare Python al lavoro.

Cosa controllare: Che proponga usi concreti e raggiungibili.

181. Roadmap personalizzata

“In base ai miei interessi (<interessi>), disegnammi una roadmap di cosa studiare dopo Python base, in ordine.”

Quando usarlo: Quando vuoi un percorso su misura.

Cosa controllare: Che la roadmap sia ordinata e parta dal tuo livello.

182. Progetti per crescere

“Quali progetti dovrei fare per passare dal livello base a quello intermedio in Python?”

Quando usarlo: Quando vuoi crescere con la pratica.

Cosa controllare: Che i progetti aumentino gradualmente di difficoltà.

183. Cosa NON serve ancora

“Cosa NON ho bisogno di studiare adesso, da principiante, anche se ne sento parlare? Così non mi disperdo.”

Quando usarlo: Quando ti senti sopraffatto da troppi argomenti.

Cosa controllare: Che ti aiuti a rimandare ciò che è prematuro.

184. Collegare Python e AI nel lavoro

“Come potrei combinare le basi di Python e l'uso dell'AI per essere più efficiente in <ambito>?”

Quando usarlo: Quando vuoi unire le due competenze.

Cosa controllare: Che proponga usi pratici e realistici.

185. Misurare se sono pronto

“Come capisco di essere pronto a passare dal livello base al successivo?”

Quando usarlo: Quando dubiti di essere pronto per andare avanti.

Cosa controllare: Che dia criteri concreti, non solo 'quando te la senti'.

186. Risorse per continuare

“Che tipo di risorse (non nomi specifici) dovrei cercare per continuare a imparare i dati con Python dopo questo libro?”

Quando usarlo: Quando vuoi sapere cosa cercare dopo.

Cosa controllare: Che indichi tipi di risorse adatti al tuo livello.

187. Dal libro al mondo reale

“Come faccio il salto da 'esercizi del libro' a 'usare Python su problemi miei reali?’”

Quando usarlo: Quando vuoi applicare ciò che sai alla realtà.

Cosa controllare: Che proponga un percorso graduale dagli esercizi ai casi veri.

188. Restare aggiornato

“Come faccio a restare aggiornato su Python e AI senza farmi travolgere, da principiante?”

Quando usarlo: Quando temi di restare indietro.

Cosa controllare: Che proponga un approccio sostenibile e selettivo.

Spiegazioni mirate sulle basi

189. Differenza print e return

“Spiegami con un esempio semplice la differenza tra print() e return in una funzione.”

Quando usarlo: Quando confondi mostrare e restituire un valore.

Cosa controllare: Che l'esempio mostri perché return serve per riusare il risultato.

190. Cosa sono i parametri

“Spiegami cosa sono i parametri e gli argomenti di una funzione, con un esempio da principiante.”

Quando usarlo: Quando le funzioni con parametri ti confondono.

Cosa controllare: Che distingua parametro (nella definizione) e argomento (nella chiamata).

191. Liste vs dizionari

“Quando devo usare una lista e quando un dizionario? Dammi un criterio semplice con esempi.”

Quando usarlo: Quando non sai quale struttura dati scegliere.

Cosa controllare: Che dia una regola pratica per scegliere.

192. for vs while

“Qual è la differenza tra un ciclo for e un ciclo while? Quando uso l'uno e quando l'altro?”

Quando usarlo: Quando non sai quale ciclo usare.

Cosa controllare: Che colleghi for a 'so quante volte' e while a 'finché' una condizione'.

193. Indentazione: la regola

“Spiegami la regola dell'indentazione in Python con esempi giusti e sbagliati.”

Quando usarlo: Quando sbagli i rientri.

Cosa controllare: Che mostri chiaramente cosa va rientrato e di quanto.

194. Cosa sono i metodi

“Cosa significa quando scrivo qualcosa.metodo(), tipo testo.upper()? Spiegamelo semplice.”

Quando usarlo: Quando vedi il punto dopo una variabile e non capisci.

Cosa controllare: Che spieghi il concetto di metodo su un oggetto.

195. None e il suo significato

“Cosa significa None in Python e quando lo incontro da principiante?”

Quando usarlo: Quando ti compare None inaspettato.

Cosa controllare: Che spieghi i casi tipici (funzione senza return, get senza default).

196. Booleani e condizioni

“Spiegami come funzionano i valori True/False e come si usano nelle condizioni, da zero.”

Quando usarlo: Quando le condizioni ti confondono.

Cosa controllare: Che colleghi i confronti ai valori booleani.

197. f-string in dettaglio

“Spiegami tutto quello che posso fare dentro le graffe di una f-string, con esempi semplici.”

Quando usarlo: Quando vuoi sfruttare bene le f-string.

Cosa controllare: Che mostri variabili, calcoli e formattazione :.2f.

198. range spiegato bene

“Spiegami range() con tutti i suoi modi d'uso (inizio, fine, passo) con esempi.”

Quando usarlo: Quando range ti dà risultati inattesi.

Cosa controllare: Che chiarisca che l'estremo destro è escluso.

199. Accumulatore: lo schema

"Spiegami lo schema dell'accumulatore (variabile che cresce in un ciclo) con un esempio passo passo."

Quando usarlo: Quando devi sommare/contare in un ciclo.

Cosa controllare: Che mostri l'inizializzazione prima del ciclo.

200. Lista di dizionari

"Spiegami cos'è una lista di dizionari e perché è utile per rappresentare dati, con un esempio."

Quando usarlo: Quando inizi a trattare dati strutturati.

Cosa controllare: Che colleghi ogni dizionario a una 'riga' di tabella.

201. try/except spiegato

"Spiegami try/except in modo semplice: a cosa serve e quando lo uso, con un esempio."

Quando usarlo: Quando vuoi gestire gli errori senza crash.

Cosa controllare: Che mostri un caso concreto (es. input non valido).

202. Conversioni di tipo

"Spiegami quando e come convertire tra testo, intero e decimale, con esempi pratici."

Quando usarlo: Quando i tipi di dato ti danno problemi.

Cosa controllare: Che chiarisca str(), int(), float() e quando servono.

202 prompt non servono a fare 202 domande: servono ad avere sempre quella giusta per il momento giusto. Tienili a portata di mano mentre studi, e usali con metodo. Nessuno nasce imparato - ma con le domande giuste si impara molto più in fretta.